Rumor Spreading in Randomized Algorithms: Theory, Variants, and Empirical Analysis

Areen Mahich and Wasi Husain

April 5, 2026

Abstract

We study the classical rumor spreading problem on complete graphs under randomized communication models such as PUSH and PULL. The process exhibits exponential growth in early stages and achieves full dissemination in $O(\log n)$ rounds with high probability. We analyze the expected spreading time, establish lower bounds, and study concentration behavior using the method of bounded differences.

We further investigate several variants including failure probabilities, multiple calls per node, and planted influencers, and compare empirical observations with theoretical predictions. In addition, we examine the algorithm of Karp et al., which maintains $O(\log n)$ dissemination time while reducing communication complexity to $O(n \log \log n)$, demonstrating the effectiveness of adaptive strategies in minimizing redundant communication.

1 Introduction

Rumor spreading is a fundamental problem in randomized distributed algorithms, with wide-ranging applications in distributed systems, peer-to-peer networks, and large-scale information dissemination. The central question is to understand how quickly information propagates through a network when nodes communicate in a randomized manner.

We consider a synchronous communication model on a complete graph, where in each round every node contacts another node chosen uniformly at random. Depending on the model (e.g., PUSH or PULL), information is either sent to or requested from the contacted node. Despite its simplicity, this process exhibits rich probabilistic behavior, including an initial phase of exponential growth followed by a saturation phase, ultimately achieving full dissemination in logarithmic time with high probability.

These models closely capture real-world systems. For example, gossip protocols in distributed databases use randomized communication to ensure fast and robust data synchronization. Similarly, information spread in social networks, viral marketing campaigns, and epidemic propagation can all be modeled using variants of rumor spreading, where individuals interact randomly and information spreads through local contacts. Understanding the efficiency and limitations of such processes is therefore crucial for designing scalable and reliable communication protocols.

2 Model and Preliminaries

We consider a complete graph on n nodes, where initially a subset of nodes is informed (typically one node). Time proceeds in discrete synchronous rounds, and in each round nodes communicate with randomly chosen neighbors.

2.1 PUSH Model

In the PUSH model, only informed nodes actively spread the rumor. In each round, every informed node selects a node uniformly at random and attempts to transmit the information.

```
Initialize: one node is informed
for each round t = 1, 2, ... do
    for each informed node u do
        Choose random node v
        v becomes informed
    end for
end for
```

This model leads to rapid exponential growth in the early stages, as the number of informed nodes doubles approximately each round.

2.2 PULL Model

In the PULL model, uninformed nodes actively try to acquire the rumor. In each round, every uninformed node contacts a random node and becomes informed if the contacted node already knows the rumor.

```
Initialize: one node is informed
for each round t = 1, 2, ... do
    for each uninformed node u do
        Choose random node v
        if v is informed then
            u becomes informed
        end if
    end for
end for
```

Unlike PUSH, the PULL model is slow in the early phase but becomes highly efficient in the later stages when most nodes are already informed.

2.3 Notation and Metrics

Let:

- X_t denote the number of informed nodes at time t
- T denote the total number of rounds required to inform all nodes

In our experiments, we measure the following key metrics:

- **Average rounds**: the average value of T over multiple independent runs
- **Total number of calls**: total communication attempts made during the process
- **Calls per node**: normalized communication cost, given by C/n
- **Useful vs. wasted calls**: fraction of calls that successfully inform a new node versus redundant calls

These metrics allow us to compare both the time efficiency and communication overhead of different rumor spreading strategies.

3 Analysis for the PUSH Model

We analyze the time T required for a rumor to spread to all n nodes in the PUSH model. Let S_t denote the set of informed nodes at time t, and $|S_t| = X_t$.

Phase-wise Growth We divide the process into two phases:

Phase 1: $X_t \leq \frac{n}{2}$ (Growth Phase).

At time t, suppose $X_t = k$. For any uninformed node, the probability it is not contacted is:

$$\left(1 - \frac{1}{n-1}\right)^k \leq e^{-k/n}$$

Thus using linearity of expectation, the expected number of newly informed nodes is:

$$\mathbb{E}[X_{t+1} - X_t \mid X_t = k] = (n-k)\left(1 - \left(1 - \frac{1}{n-1}\right)^k\right)$$

For $k \leq n/2$, this implies:

$$\mathbb{E}[X_{t+1} - X_t \mid X_t = k] \geq \frac{k}{4} \Rightarrow \mathbb{E}[X_{t+1} \mid X_t = k] \geq \frac{5}{4}\,k$$

From which we get the recursive relation:

$$\mathbb{E}[X_{t+1}] = \sum_k \Pr(X_t = k) \times \mathbb{E}[X_{t+1} \mid X_t = k] \geq \frac{5}{4}\sum_k \Pr(X_t = k) \times k = \frac{5}{4}\mathbb{E}[X_t]$$

Hence, the number of informed nodes grows exponentially:

$$\mathbb{E}[X_t] \geq \left(\frac{5}{4}\right)^t$$

This phase completes in:

$$T_1 = O(\log n)$$

Phase 2: $X_t \geq \frac{n}{2}$ (Saturation Phase).

Let $U_t = n - X_t$ be the number of uninformed nodes. For any uninformed node:

$$\Pr(\text{not informed in one round}) = \left(1 - \frac{1}{n-1}\right)^k \leq e^{-1/2}$$

Thus we get the recursive relation:

$$\mathbb{E}[U_{t+1} \mid U_t = k] \leq e^{-1/2}k$$

So the number of uninformed nodes decays exponentially:

$$\mathbb{E}[U_t] \leq \frac{n}{2} \cdot e^{-t/2}$$

This phase completes in:

$$T_2 = O(\log n)$$

Total Expected Time Combining both phases:

$$\mathbb{E}[T] = T_1 + T_2 = \Theta(\log n)$$

Concentration Around the Mean We now show that the completion time T is tightly concentrated around its expectation.

Let Y_i denote the number of newly informed nodes in round i, and fix outcomes $Y_1 = y_1, \ldots, Y_{i-1} = y_{i-1}$. Let S_i be the informed set at the beginning of round i with $|S_i| = k$.

We analyze the sensitivity of T to changes in a single round outcome. Consider two possible values y_i and y_i' for Y_i. These lead to different states:

$$|S_{i+1}| = k + y_i \quad \text{and} \quad |S'_{i+1}| = k + y_i'$$

Since $0 \leq Y_i \leq \min(k, n-k)$, we have:

$$k \leq |S_{i+1}| \leq \min(2k, n)$$

Thus, the maximum deviation occurs between the extreme cases:

$$|S_{i+1}| = k \quad \text{and} \quad |S_{i+1}| = \min(2k, n)$$

Hence, (letting $\mathcal{F}_j :=$ the event that $Y_1 = y_1, ..., Y_j = y_j$)

$$\big|\mathbb{E}[T \mid Y_i = y_i, \mathcal{F}_{i-1}] - \mathbb{E}[T \mid Y_i = y_i', \mathcal{F}_{i-1}]\big| \leq \Delta_i$$

where

$$\Delta_i = \mathbb{E}[T \mid |S_{i+1}| = k] - \mathbb{E}[T \mid |S_{i+1}| = \min(2k, n)]$$

Case 1: $2k < n$ (growth phase).
Note that after the i_{th} day, the expected growth will follow the same exponential pattern as shown in growth and saturation phases
From the exponential growth bounds:

$$\mathbb{E}[T \mid |S_{i+1}| = k] = i + 1 + \Theta\big(\log(n/k)\big)$$

$$\mathbb{E}[T \mid |S_{i+1}| = 2k] = i + 1 + \Theta\big(\log(n/2k)\big)$$

Thus,

$$\Delta_i = \Theta\big(\log(n/k) - \log(n/2k)\big) = O(1)$$

Case 2: $2k \geq n$ (saturation phase).
In this scenario $\mathbb{E}[T \mid |S_{i+1}| = n] = i + 1$

Let $U_{i+1} = n - |S_{i+1}|$. From the exponential decay of uninformed nodes:

$$\mathbb{E}[U_{i+t}] \leq U_{i+1} \cdot e^{-t/2}$$

Using a union bound over uninformed nodes, the probability that the process continues for t more rounds decays exponentially:

$$\Pr(T > i + t \mid |S_{i+1}| = k) \leq (n-k)e^{-t/2}$$

Summing tail probabilities gives:

$$\mathbb{E}[T \mid |S_{i+1}| = k] \leq i + 1 + O(1)$$

Hence in this case also, $\Delta_i = O(1)$.

Bounded Differences.

Therefore, for every round i, there exists a constant $c_i = O(1)$ such that:

$$\left|\mathbb{E}[T \mid Y_i = y_i, \mathcal{F}_{i-1}] - \mathbb{E}[T \mid Y_i = y_i', \mathcal{F}_{i-1}]\right| \leq c_i$$

Since the process completes in $O(\log n)$ rounds, we have:

$$\sum_i c_i^2 = O(\log n)$$

Applying the inequality:

$$\Pr\left(|T - \mathbb{E}[T]| > t\right) \leq 2\exp\left(-\frac{t^2}{c\log n}\right)$$

for some constant $c > 0$.

Conclusion Thus, $T = \Theta(\log n)$ with high probability, and deviations larger than $\sqrt{\log n}$ are exponentially unlikely.

4 Analysis for the PULL Model

We analyze the time T required for a rumor to spread to all n nodes under the **PULL** model.

Let S_t denote the set of informed nodes at time t, and let $|S_t| = X_t$. Let $U_t = n - X_t$ denote the number of uninformed nodes.

In the PULL model, in each round, every uninformed node contacts a uniformly random node and becomes informed if the contacted node is informed.

Phase-wise Analysis

We divide the process into two phases.

Phase 1: $X_t \leq \frac{n}{2}$ (Growth Phase).

Suppose $X_t = k$. Consider any uninformed node x. The probability that x becomes informed in this round is:

$$\Pr(x \text{ gets informed}) = \frac{k}{n-1}$$

Thus, the expected number of newly informed nodes is:

$$\mathbb{E}[X_{t+1} - X_t \mid X_t = k] = (n-k)\frac{k}{n-1}$$

Since $k \leq n/2$, we have $n - k \geq n/2$, hence:

$$\mathbb{E}[X_{t+1} - X_t \mid X_t = k] \geq \frac{n}{2} \cdot \frac{k}{n-1} \geq \frac{k}{2} \cdot \frac{n}{n-1} \geq \frac{k}{2}$$

Therefore:

$$\mathbb{E}[X_{t+1} \mid X_t = k] \geq \frac{3}{2}k$$

Taking expectation over X_t:

$$\mathbb{E}[X_{t+1}] \geq \frac{3}{2}\mathbb{E}[X_t]$$

Solving the recurrence:

$$\mathbb{E}[X_t] \geq \left(\frac{3}{2}\right)^t$$

Thus, Phase 1 completes in:

$$T_1 = O(\log n)$$

Phase 2: $X_t \geq \frac{n}{2}$ (Saturation Phase)

Let $U_t = n - X_t$ be the number of uninformed nodes.

Consider any uninformed node x. The probability that it *remains uninformed* in a round is:

$$\Pr(x \text{ remains uninformed}) = 1 - \frac{k}{n-1}$$

Since $k \geq n/2$, we have:

$$\Pr(x \text{ remains uninformed}) \leq 1 - \frac{1}{2} = \frac{1}{2}$$

Thus:

$$\mathbb{E}[U_{t+1} \mid U_t = k] \leq \frac{1}{2}k$$

Taking expectations:

$$\mathbb{E}[U_{t+1}] \leq \frac{1}{2}\mathbb{E}[U_t]$$

Solving:

$$\mathbb{E}[U_t] \leq \frac{n}{2} \cdot 2^{-t}$$

Thus Phase 2 completes in:

$$T_2 = O(\log n)$$

Total Expected Time

Combining both phases:

$$\mathbb{E}[T] = T_1 + T_2 = O(\log n)$$

5 PUSH vs PULL

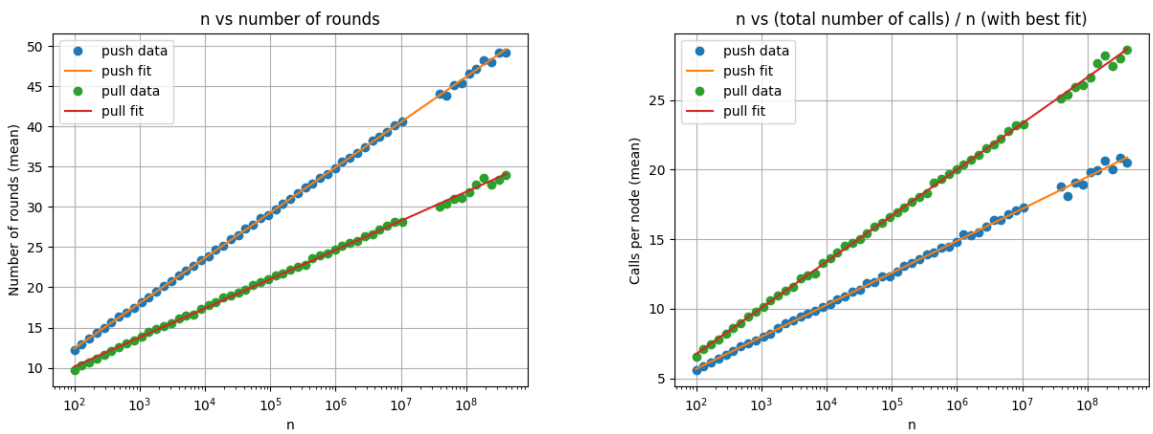


Figure 1: Comparison of PUSH and PULL strategies.

The difference between PUSH and PULL can be understood by examining the spreading process across different phases and how each model distributes efficiency over time. In the PUSH model, the number of informed nodes grows approximately exponentially in the initial phase, leading to rapid expansion. However, in the later stages, when most nodes are already informed, a large fraction of calls collide with already informed nodes. Informing the remaining few nodes becomes increasingly difficult, resulting in a long tail. This phase dominates the total completion time, as progress per round becomes very small despite many calls being made.

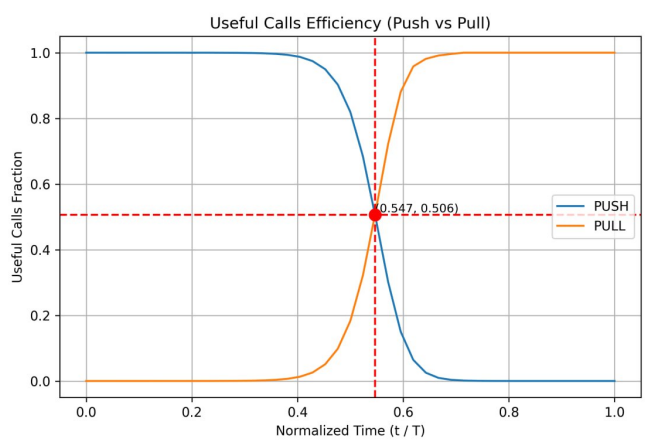


Figure 2: Comparison of PUSH, PULL, and hybrid strategies.

In contrast, the PULL model exhibits the opposite behavior. In the initial phase, PULL is inefficient since each uninformed node contacts a random node and succeeds with low probability, leading to a large number of wasted calls. These early rounds dominate the total communication cost, as almost all n nodes are actively making calls with very few successful transmissions. However, as the number of informed nodes increases, the success probability rises sharply. In the final phase, when only a small number of nodes remain uninformed, each such node becomes informed with high

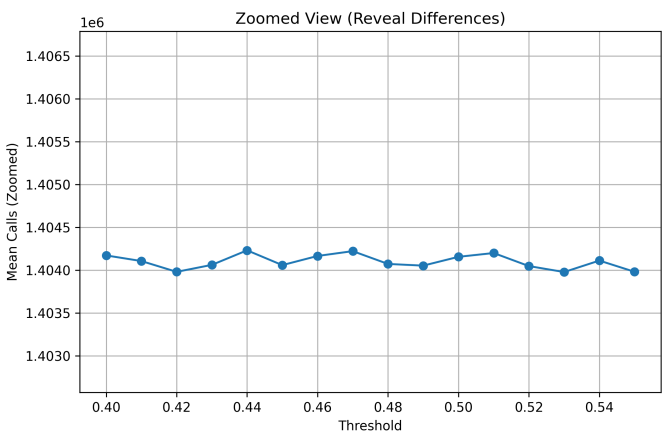
probability in each round. This leads to a rapid, near geometric decay in the number of uninformed nodes, allowing the process to finish quickly.

Although both PUSH and PULL complete in $O(\log n)$ rounds, their constant factors differ due to this asymmetry. PUSH requires additional rounds in the final phase (analogous to the coupon collector effect), leading to a total of approximately $\log_2 n + \ln n$ rounds. In contrast, PULL completes in approximately $\log_2 n + \log\log n$ rounds, as it resolves the tail much more efficiently. Thus, PULL achieves faster completion despite its poor initial performance.

This difference in time is complemented by the opposite behavior in communication cost. PUSH incurs most of its inefficiency in the final phase, resulting in about $n \ln n$ total calls. On the other hand, PULL is highly inefficient in the initial phase, where nearly all nodes are making calls for several rounds, leading to approximately $n \log_2 n$ total calls. Since $\ln n < \log_2 n$, PUSH ultimately results in fewer total calls.

These observations highlight a fundamental trade-off: PUSH is communication-efficient but slower to finish, while PULL is faster but incurs higher communication overhead. This naturally motivates a hybrid approach that combines the strengths of both models—using PUSH in the early phase to exploit exponential growth, and switching to PULL in the later phase to eliminate the long tail efficiently.

A natural transition point occurs when the efficiencies of PUSH and PULL become comparable, which happens when roughly half the nodes are informed. To explore this, we experimented with a hybrid strategy that switches from PUSH to PULL once a fraction αn of nodes are informed. However, fine-tuning this parameter is challenging in practice. The number of informed nodes can change significantly within a single round (for example, from 49% to over 55%), making it difficult to switch precisely at a desired threshold. As a result, the performance is relatively insensitive to α in the range 0.5 to 0.6, leading to a flat region in the empirical results.



Furthermore, such a threshold-based strategy is impractical in real-world settings, as nodes typically do not have access to global information such as the total number of informed nodes. This makes it difficult to coordinate a precise global transition. These limitations motivate the need for adaptive hybrid strategies that rely only on local information. The algorithm of Karp et al. achieves this by using simple local counters and rules to decide when to change behavior or stop participating, effectively approximating the optimal transition without requiring global knowledge.

6 Hybrid Push-Pull Approach (Karp et al.)

6.1 Motivation for a Hybrid Model

The PUSH model is highly effective in the early stages of rumor spreading, where the number of informed nodes is small and most contacts are useful. However, as the process progresses and a large fraction of nodes become informed, PUSH suffers from significant inefficiency due to repeated contacts with already informed nodes.

Conversely, the PULL model performs poorly in the early phase but becomes highly efficient in the later stages, as uninformed nodes actively seek information from a largely informed population.

This naturally motivates a hybrid approach that combines the strengths of both models. However, in real-world decentralized systems, nodes do not have global knowledge of the system state (e.g., how many nodes are informed). Therefore, it is not feasible to explicitly switch from PUSH to PULL based on a global condition.

6.2 Key Idea

The algorithm proposed by Richard Karp et al. introduces a fully distributed mechanism where nodes adapt their behavior based on local observations. Instead of relying on global thresholds, each node transitions between different states using simple counters and interaction patterns.

The central idea is to:

- Use PUSH to rapidly grow the set of informed nodes
- Detect when spreading becomes inefficient
- Gradually reduce redundant communication
- Ensure completion using controlled PULL-like behavior

6.3 State-Based Design

Each node maintains a local state, which governs its behavior:

- **State A (Uninformed):** Node has not received the rumor.
- **State B (Active):** Node actively participates in spreading (push and pull).
- **State C (Final Phase):** Node continues spreading for a limited number of rounds.
- **State D (Inactive):** Node stops participating to avoid redundant communication.

6.4 State Transitions

State transitions are based on local interactions:

- A $\rightarrow$ B: When a node first receives the rumor.
- B $\rightarrow$ B: Continues active spreading while new nodes are frequently discovered.
- B $\rightarrow$ C: Triggered when repeated interactions suggest most neighbors are already informed.
- C $\rightarrow$ D: After a fixed number of rounds (typically $O(\log \log n)$).

This mechanism allows nodes to implicitly detect the transition from the exponential phase to the saturation phase without any global coordination.

6.5 Intuition

The algorithm mimics real-world information dissemination:

- Early phase: aggressive broadcasting (push)
- Mid phase: mixed behavior
- Late phase: targeted completion (pull-like)

By reducing redundant communication in the later stages, the algorithm achieves the same $O(\log n)$ time as PUSH, while reducing total communication to $O(n \log \log n)$.

6.6 Pseudo-code (Simplified)

```
for each node u:
    state[u] = A
    counter[u] = 0

if u initially has rumor:
    state[u] = B

for each round:
    for each node u:

        if state[u] == A:
            v = random_node()
            if v has rumor:
                state[u] = B

        else if state[u] == B:
            v = random_node()

            // push
            send rumor to v

            // pull
            if v has rumor:
                counter[u]++

            if counter[u] exceeds threshold:
                state[u] = C

        else if state[u] == C:
            v = random_node()
            send rumor to v

            counter[u]++

            if counter[u] exceeds loglog(n):
```

```
            state[u] = D

        else if state[u] == D:
            // inactive
            do nothing
```

6.7 Discussion

This adaptive strategy allows the system to automatically transition between different phases of spreading. Unlike naive PUSH or PULL, it avoids excessive redundant communication while preserving fast dissemination time. The key strength lies in its ability to approximate global behavior using only local information.

7 Variants

We study several natural extensions of the basic rumor spreading model to understand how different system parameters affect dissemination time and communication efficiency.

7.1 Failure Probability

In this variant, each communication attempt succeeds independently with probability $1 - p$. A failed call does not transmit the rumor, effectively reducing the rate at which information spreads.

The key question is how the failure probability p impacts the overall dissemination time. Intuitively, as p increases, fewer successful transmissions occur per round, slowing down the growth of informed nodes. While the spreading process still exhibits logarithmic behavior in n, the effective rate is reduced, leading to an expected runtime of the form:

$$T = \Theta\left(\frac{\log n}{1 - p}\right)$$

Thus, higher failure probabilities increase the constant factor, and as $p \to 1$, the process becomes extremely slow.

This models real-world scenarios such as unreliable communication networks, packet loss in distributed systems, or misinformation being ignored in social networks.

7.2 Multiple Calls per Node

In this setting, each informed node contacts k random nodes in every round instead of just one. This increases the parallelism of the spreading process.

The hypothesis is that increasing k accelerates dissemination, as more nodes are contacted per round. The expected runtime becomes:

$$T = O\left(\frac{\log n}{\log(1 + k)}\right)$$

indicating that higher k leads to faster spreading due to a larger effective branching factor.

However, this comes at the cost of increased communication overhead, as the total number of calls per round grows linearly with k. Thus, there is a trade-off between speed and communication cost.

This variant captures real-world systems where nodes can broadcast or contact multiple peers simultaneously, such as gossip protocols in distributed systems or viral content shared across multiple channels.

7.3 Planted Influencers

Here, instead of starting with a single informed node, we initialize the process with multiple informed nodes (influencers). These nodes act as parallel sources of information propagation.

The key question is how the number of initial sources affects convergence time. Empirically, increasing the number of influencers significantly reduces the spreading time, often leading to behavior closer to:

$$T = O(\log \log n)$$

for sufficiently large initial sets.

This improvement occurs because the initial phase of exponential growth is effectively bypassed, allowing the rumor to reach a large fraction of the network much earlier.

This models real-world phenomena such as coordinated information campaigns, influencer marketing, or multiple entry points of information in a network, where parallel dissemination dramatically accelerates reach.

8 Experimental Setup

We implemented large-scale simulations of rumor spreading under multiple models, with a focus on scalability, statistical reliability, and reproducibility. Our design choices were guided by the need to simulate networks up to $n \approx 10^8$–10^9 while maintaining both accuracy and computational efficiency.

8.1 Simulation Framework

All experiments are performed in a synchronous setting. Initially, a subset of nodes is informed (typically one node), and in each round nodes perform randomized communication (PUSH, PULL, or their variants). The state of each node is tracked using a compact boolean representation indicating whether it is informed.

To avoid interference within a round, newly informed nodes are buffered and only added to the active set at the end of each round. This ensures correctness of the discrete-time model and avoids artificial acceleration of the spreading process.

8.2 Randomness and Sampling

Efficient and high-quality randomness is critical for accurate simulation. We use the `xoshiro256++` pseudo-random number generator, seeded via `SplitMix64`. This combination provides both strong statistical guarantees and extremely fast generation speed, which is essential since billions of random samples are required across all experiments.

Alternative generators such as `mt19937` were considered, but rejected due to higher computational overhead. Simpler generators (e.g., linear congruential methods) were avoided due to poor statistical properties, which could bias the spreading dynamics.

8.3 Parameter Scaling Strategy

To study asymptotic behavior, we vary the network size n multiplicatively, starting from $n = 100$ and increasing up to 10^8–10^9. Specifically, we use a growth factor in the range 1.2–1.4, which provides a good trade-off between resolution and computational cost.

This multiplicative scaling allows us to capture logarithmic trends effectively while avoiding an excessive number of data points. Additive scaling was avoided as it would require significantly more samples to observe asymptotic behavior.

8.4 Adaptive Iteration Strategy

For each value of n, multiple independent trials are performed and averaged. The number of iterations is chosen adaptively based on variance:

- For small n, variance is higher, so we perform a larger number of iterations.
- For large n, strong concentration reduces variance, allowing fewer iterations.

This approach ensures statistical reliability while keeping total runtime manageable. A fixed number of iterations for all n was avoided, as it would either waste computation (for large n) or produce noisy results (for small n).

8.5 Memory and Data Structures

To support large-scale simulations, we use compact data structures:

- Boolean bit arrays to store node states (instead of full integers)
- Preallocated vectors for informed nodes to avoid dynamic resizing
- Batch updates of newly informed nodes

This reduces memory usage by a constant factor and improves cache locality. More complex structures such as sets or maps were avoided due to higher overhead and poor cache performance. While bitsets or SIMD-based approaches could further optimize performance, they introduce additional implementation complexity and were not required for our scale.

8.6 Metrics Collected

We record the following metrics:

- Total number of rounds required for full dissemination
- Total number of communication calls
- Calls per node (C/n)
- Fraction of informed nodes over time
- Fraction of useful vs. wasted calls

These metrics allow us to evaluate both time complexity and communication efficiency across different models.

8.7 Variant-Specific Setup

8.7.1 Failure Probability

In this variant, each communication succeeds independently with probability $1 - f$. Failed calls do not result in information transfer. We vary f across a wide range and measure its impact on dissemination time. This isolates the effect of unreliable communication on spreading dynamics.

8.7.2 Multiple Calls per Node

Each informed node contacts k random nodes per round. We vary k to study how increased parallelism affects spreading time. While larger k accelerates dissemination, it also increases communication overhead, allowing us to explore the trade-off between speed and cost.

8.7.3 Planted Influencers

Instead of a single source, we initialize multiple informed nodes (influencers). The number of initial sources is varied to observe how parallelization of the early phase impacts convergence time. This setup highlights the reduction in spreading time due to bypassing the initial growth phase.

8.7.4 Karp et al. Algorithm

We implement the algorithm of Karp et al., which combines push and pull strategies with adaptive behavior to reduce redundant communication. In this setup, nodes switch strategies based on the phase of the process. We measure both dissemination time and total number of calls, enabling direct comparison with standard PUSH and PULL models.

Overall, these design choices enable us to perform large-scale, high-fidelity simulations that capture both asymptotic trends and constant-factor effects across different rumor spreading variants.

9 Results and Discussion

We empirically evaluate rumor spreading under different models and compare the observed behavior with theoretical predictions.

9.1 Effect of Failure Probability

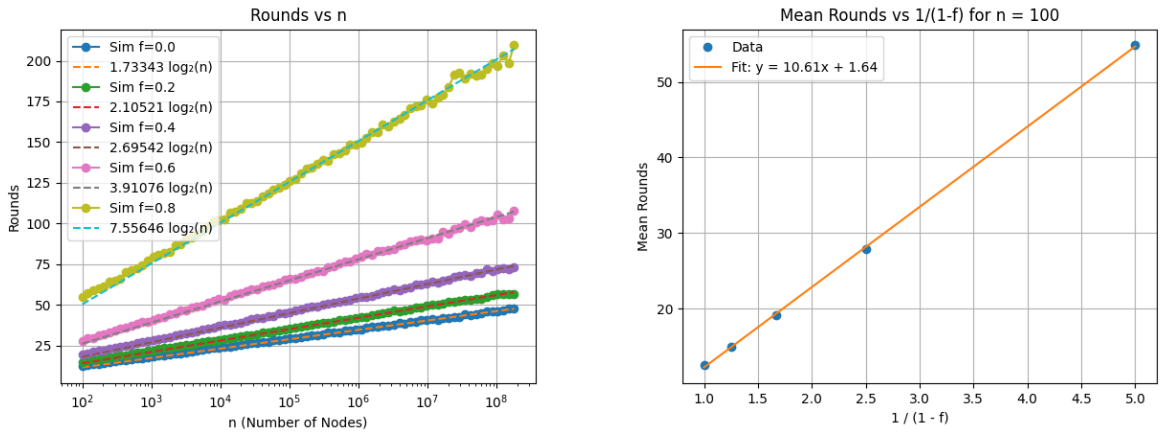


Figure 3: Left:Avg no of rounds vs network size n. Right:Avg no of rounds vs failure probability f

Introducing a failure probability f slows down the dissemination process by reducing the success rate of each communication. In the left plot (Rounds vs n), the spreading time continues to scale logarithmically with n for all values of f, but with increasing slopes as f grows. This indicates a larger constant factor in the $O(\log n)$ behavior, reflecting reduced efficiency due to failed transmissions.

In the right plot, with n fixed, the number of rounds increases sharply as f increases, following a trend close to $1/(1-f)$. This highlights an inverse relationship between the success probability $(1-f)$ and the spreading time. As f approaches 1, successful communication becomes rare, leading to a rapid slowdown.

Overall, while the logarithmic growth persists, higher failure probabilities significantly degrade performance, emphasizing the importance of reliable communication.

9.2 K calls per node

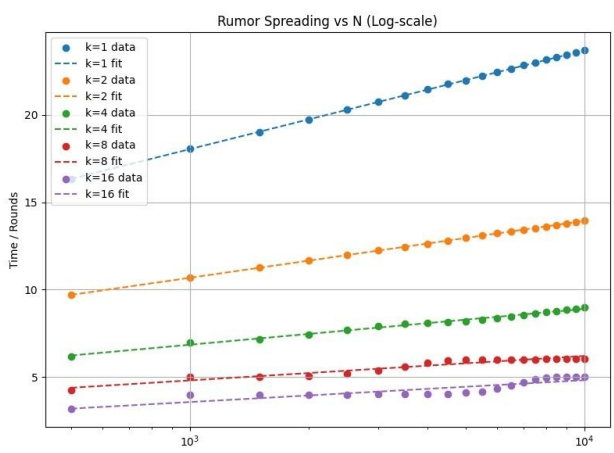


Figure 4: Avg no of rounds vs network size f.

Increasing the number of calls per node (k) significantly improves the speed of rumor dissemination. From the plotted data, we observe that the number of rounds required continues to scale logarithmically with the network size N for all values of k, i.e., $T(N) = O(\log N)$, but with different constant factors.

The best-fit lines for different values of k are:

$$\begin{aligned} k = 1: &\quad T(N) \approx 2.4547 \log N + 1.0796 \\ k = 2: &\quad T(N) \approx 1.4145 \log N + 0.9055 \\ k = 4: &\quad T(N) \approx 0.8909 \log N + 0.6918 \\ k = 8: &\quad T(N) \approx 0.6080 \log N + 0.6055 \\ k = 16: &\quad T(N) \approx 0.5437 \log N - 0.1883 \end{aligned}$$

As k increases, the slope of the logarithmic fit decreases, indicating faster dissemination. This is expected since each node contacts more neighbors per round, increasing parallelism in spreading the rumor.

However, the improvement exhibits diminishing returns: the reduction in slope becomes smaller as k grows (e.g., from $k = 8$ to $k = 16$). This suggests that beyond a certain point, increasing k yields limited additional benefit relative to the increased communication cost.

Overall, increasing k improves performance by reducing the constant factor in the $O(\log N)$ complexity, while preserving the same asymptotic behavior.

9.3 Planted Influencers

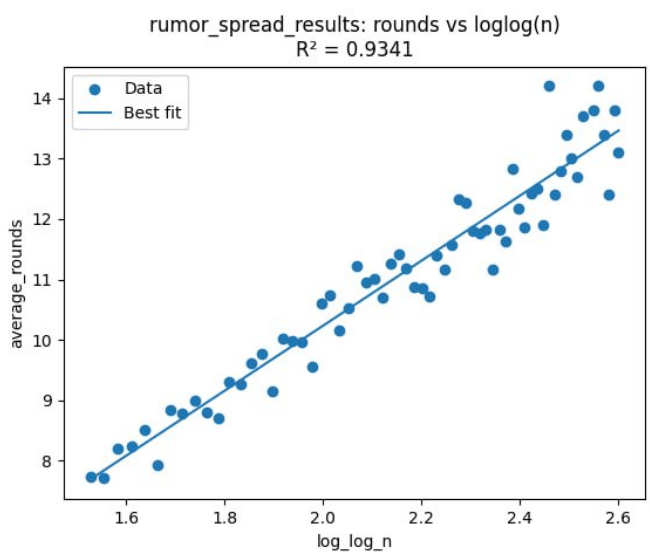


Figure 5: Dissemination time vs number of initial informed nodes.

When multiple nodes are initially informed (planted influencers), the dissemination time reduces significantly. As shown in the plot, the average number of rounds grows much slower compared to the single-source case, and empirically fits a function of the form $T(n) \approx 5.3777 \log \log n - 0.5211$. This indicates a transition from $O(\log n)$ behavior to $O(\log \log n)$ scaling as the number of initial sources increases.

The improvement arises because multiple influencers parallelize the early stages of spreading, effectively bypassing the initial exponential growth phase. As a result, the rumor reaches a large fraction of the network much faster, leading to a substantial reduction in overall dissemination time.

9.4 Karp et al. Algorithm

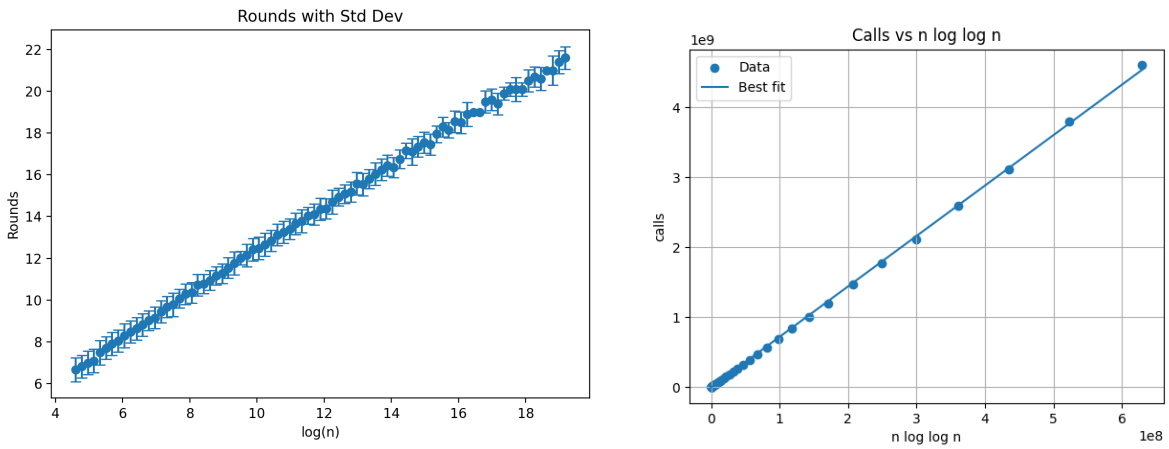


Figure 6: Left: Rounds vs n. Right: Total calls vs n for Karp algorithm.

The Karp et al. algorithm achieves $O(\log n)$ dissemination time while significantly reducing communication overhead. As observed in the plot, the number of rounds scales logarithmically with n, with an empirical linear fit (slope ≈ 7.22, $R^2 \approx 0.9998$) confirming strong agreement with $O(\log n)$ behavior.

In contrast to the standard PUSH/PULL approaches, the total number of calls grows much slower, aligning with the expected $O(n \log \log n)$ complexity. This demonstrates that the algorithm maintains fast spreading while reducing redundant communication, highlighting the effectiveness of adaptive strategies in later stages of the process.

9.5 Discussion

Across all experiments, we observe strong agreement with theoretical predictions. The exponential growth phase is clearly visible in early rounds, followed by a slowdown near $n/2$ due to increased redundancy.

Variants such as failures and multiple initial sources primarily affect constants rather than asymptotic behavior, while adaptive strategies like PUSH-PULL and the Karp algorithm significantly improve communication efficiency.

These results highlight the importance of phase-aware and adaptive mechanisms in optimizing rumor spreading processes.

10 Limitations and Possible Improvements

While the classical PUSH and PULL models achieve optimal logarithmic dissemination time, they exhibit several inefficiencies that arise from their simplicity. These inefficiencies suggest natural directions for improvement.

10.1 Uniform Random Sampling

Both PUSH and PULL rely on uniform random sampling of communication partners. While this ensures simplicity and symmetry, it also leads to inefficiencies. Nodes select targets without any knowledge of their state, resulting in redundant interactions. In the PUSH model, this leads to a large number of collisions in the later stages, while in the PULL model, it results in many unsuccessful attempts during the early phase. This lack of bias prevents the system from prioritizing useful communication.

10.2 Memoryless Behavior

Another key limitation is that nodes operate in a memoryless manner. Each node makes decisions independently in every round without retaining any information about past interactions. As a result, nodes may repeatedly contact the same peers or fail to adapt based on previous successes or failures. This leads to unnecessary communication overhead and prevents the system from improving efficiency over time.

10.3 Lack of Global Awareness

The behavior of nodes does not depend on the global state of the system, such as the fraction of informed nodes. Consequently, nodes cannot adapt their strategy based on the phase of the spreading process. For example, an optimal strategy would use PUSH in the early phase and PULL in the later phase, but implementing such a transition requires knowledge of global progress, which is not available in the basic model.

10.4 Adaptive Sampling and Local Learning

A natural improvement is to introduce adaptive behavior based on local information. Instead of sampling uniformly at random, nodes could bias their choices based on past interactions or observed success rates. For instance, nodes may avoid repeatedly contacting the same peers or adjust their strategy when the probability of successful transmission decreases. Even simple local statistics, such as the ratio of successful to failed calls, can provide useful signals for adaptation.

10.5 Decentralized Phase Detection

Rather than relying on global thresholds, nodes can attempt to infer the phase of the process using local observations. For example, a node experiencing a high success rate may continue using PUSH, while a node observing frequent failures may switch to PULL. Such decentralized mechanisms can approximate the optimal transition point without requiring global coordination.

10.6 Connection to Adaptive Algorithms

These ideas are formalized in the algorithm of Karp et al., which combines PUSH and PULL strategies using simple local rules and counters. By allowing nodes to adapt based on local information, the algorithm reduces redundant communication while maintaining fast dissemination. This demonstrates that incorporating limited memory and adaptive behavior can significantly improve the efficiency of rumor spreading protocols.

11 Applications

Rumor spreading algorithms are widely used in systems that require fast and robust information dissemination.

Distributed Systems (Gossip Protocols): In large-scale distributed systems, gossip protocols use rumor spreading to propagate updates such as state information or membership changes. Each node communicates with randomly selected peers, ensuring that updates reach all nodes in $O(\log n)$ rounds while maintaining scalability and fault tolerance.

Blockchain and Peer-to-Peer Networks: In blockchain networks, transactions and blocks are disseminated using mechanisms similar to rumor spreading. Nodes forward newly received information to random peers, enabling rapid propagation across the network. However, the saturation phase leads to redundant transmissions, which motivates optimized gossip strategies to reduce bandwidth overhead.

Epidemic Modeling: Rumor spreading closely models the spread of infectious diseases, where individuals correspond to nodes and infection corresponds to information spread. The initial exponential growth and later saturation phases mirror real epidemic curves, helping in predicting spread dynamics and designing control strategies.

Other Applications:

- Content distribution in peer-to-peer systems
- Sensor networks and data aggregation
- Load balancing in distributed systems
- Social network information diffusion

12 Conclusion

In this work, we studied the rumor spreading problem through both theoretical analysis and empirical evaluation. We observed that simple randomized strategies such as PUSH, PULL, and their hybrids achieve efficient information dissemination in $O(\log n)$ rounds, with strong concentration around the expected behavior. The phase-wise analysis provided insight into the dynamics of spreading, particularly the transition from exponential growth to saturation. Furthermore, exploring variants such as failure probabilities, multiple calls, and influencers highlighted the robustness and limitations of these algorithms in practical settings. Overall, rumor spreading demonstrates how simple local interactions can lead to fast and reliable global communication, making it a powerful paradigm for modern distributed systems.

Acknowledgment

We sincerely thank Prof. Surender Baswana for his constant support and for encouraging open exploration throughout the project. His valuable insights, guidance, and thought-provoking directions greatly helped shape our understanding and approach.

References

[1] R. Karp, C. Schindelhauer, S. Shenker, B. Vocking, "Randomized rumor spreading," in *Proceedings of the 41st Annual Symposium on Foundations of Computer Science (FOCS)*, 2000.

[2] N. Fountoulakis, K. Panagiotou, and T. Sauerwald, "Ultra-Fast Rumor Spreading in Social Networks," *SIAM Journal on Discrete Mathematics*, 2012.

[3] R. Motwani and P. Raghavan, *Randomized Algorithms*, Cambridge University Press, 1995.

[4] S. Baswana, *Lecture Notes on Randomized Algorithms*, IIT Kanpur.

[5] A. Mahich and W Husain, "Rumor Spreading Experiments," GitHub Repository, 2026. Available at: `https://github.com/areenm23/CS648`